nmslib / hnswlib

Code Issues 242 Pull requests 70 Actions Projects Security and quality Insights

hnswlib

Public

Watch 70 Fork 864

About

Header-only C++/python library for fast approximate nearest neighbors

github.com/nmslib/hnswlib

Readme
License
Activity Custom properties
5.4k stars 70 watching 864 forks
Report repository

m... 20 Branches 13 Tags Go to file T Go to file Add file Code

ilyajob05 Merge pull request #682 from nmslib/f/v0.10.0-rc.2 ca42672 · last month

.github/workflows	No-exceptions APIs and CI/CD improveme...	last month
examples	No-exceptions APIs and CI/CD improveme...	last month
hnswlib	Prepare v0.10.0-rc.2: fix rc.1 review blocke...	last month
python_bindings	No-exceptions APIs and CI/CD improveme...	last month
tests	Prepare v0.10.0-rc.2: fix rc.1 review blocke...	last month
.gitignore	No-exceptions APIs and CI/CD improveme...	last month
ALGO_PARAMS.md	chore(ALGO_PARAMS.md): Fix typo	4 years ago
CMakeLists.txt	Prepare v0.10.0-rc.2: fix rc.1 review blocke...	last month
LICENSE	Initial commit	9 years ago
MANIFEST.in	Add license file to pypi package	6 years ago
Makefile	Refactoring of project structure (#432)	4 years ago
README.md	Prepare v0.10.0-rc.2: fix rc.1 review blocke...	last month
TESTING_RECALL.md	Highlight code	4 years ago
pyproject.toml	PEP-517 support	6 years ago
setup.py	Prepare v0.10.0-rc.2: fix rc.1 review blocke...	last month
test_all_build_types.sh	No-exceptions APIs and CI/CD improveme...	last month

README License

Hnswlib - fast approximate nearest neighbor search

Header-only C++ HNSW implementation with python bindings, insertions and updates.

NEWS:

version 0.10.0rc2 (release candidate — not a stable PyPI upload)

- Package version is `0.10.0rc2` (PEP 440); do not upload as final `0.10.0` yet. Supersedes v0.10.0-rc.1.
- Stream `loadIndexNoExceptions` fails closed on an unopened/failed input without clearing a live index
- `searchKnnCloserFirst` is `const` again; `addPoint(..., int level)` is restored so integer levels are not treated as `replace_deleted`
- CMake no longer wipes caller `CMAKE_CXX_FLAGS` (including `add_subdirectory` / examples-off)
- `StatusOr` stores T inline and moves the result; `*NoExceptions` stream writes return `Status` instead of throwing on write failure
- Optional no-exceptions C++ API: `*NoExceptions` methods return `Status` / `StatusOr` so the headers can be compiled with `-fno-exceptions` (`-DHNSWLIB_ENABLE_EXCEPTIONS=OFF`). Throwing methods remain the default. (#619, #678) by @michaelbautin
- Stream `saveIndex` / `loadIndex` over `std::ostream` / `std::istream` , plus `getInternalIdByLabel`
- CI covers exceptions on/off, Clang / GCC / MSVC, and ASAN / UBSAN
- Fixed Clang UBSan misaligned label store in addPoint
- Report addPoint capacity and stream write errors via Status

version 0.9.0

- Fixed incorrect results in bruteforce search with filter (#514) by @lukaszsmolinski
- Fixed missing normalization check in BFIndex (#514) by @lukaszsmolinski
- Throw an exception when fewer than k elements are available (#514) by @lukaszsmolinski
- Remove unused variable (#531) by @lulyon
- Change cosine similarity to distance in README by @yurymalkov

version 0.8.0

- Multi-vector document search and epsilon search (for now, only in C++)
- By default, there is no statistic aggregation, which speeds up the multi-threaded search (it does not seem like people are using it anyway: Issue #495).
- Various bugfixes and improvements
- `get_items` now have `return_type` parameter, which can be either 'numpy' or 'list'

Full list of changes: #523

version 0.7.0

- Added support to filtering (#402, #430) by @kishorenc
- Added python interface for filtering (though note its performance is limited by GIL) (#417) by @gtsoukas
- Added support for replacing the elements that were marked as delete with newly inserted elements (to control the size of the index, #418) by @dyashuni
- Fixed data races/deadlocks in updates/insertion, added stress test for multithreaded operation (#418) by @dyashuni
- Documentation, tests, exception handling, refactoring (#375, #379, #380, #395, #396, #401, #406, #404, #409, #410, #416, #415, #431, #432, #433) by @jlmelville, @dyashuni, @kishorenc, @korzhenevski, @yoshoku, @jianshu93, @PLNech
- global linkages (#383) by @MasterAler, USE_SSE usage in MSVC (#408) by @alxvth

Highlights:

1. Lightweight, header-only, no dependencies other than C++ 11
2. Interfaces for C++, Python, external support for Java and R (https://github.com/jlmelville/rcpphnsw).
3. Has full support for incremental index construction and updating the elements (thanks to the contribution by Apoorv Sharma). Has support for element deletions (by marking them in index, later can be replaced with other elements). Python index is picklable.
4. Can work with custom user defined distances (C++).
5. Significantly less memory footprint and faster build time compared to current nmslib's implementation.

Description of the algorithm parameters can be found in ALGO_PARAMS.md.

Python bindings

Supported distances:

Distance	parameter	Equation
Squared L2	'l2'	d = sum((Ai-Bi)^2)
Inner product	'ip'	d = 1.0 - sum(Ai*Bi)
Cosine distance	'cosine'	d = 1.0 - sum(Ai*Bi) / sqrt(sum(Ai*Ai) * sum(Bi*Bi))

Note that inner product is not an actual metric. An element can be closer to some other element than to itself. That allows some speedup if you remove all elements that are not the closest to themselves from the index.

For other spaces use the nmslib library https://github.com/nmslib/nmslib.

API description

- `hnswlib.Index(space, dim)` creates a non-initialized index an HNSW in space `space` with integer dimension `dim`.

`hnswlib.Index` methods:

- `init_index(max_elements, M = 16, ef_construction = 200, random_seed = 100, allow_replace_deleted = False)` initializes the index from with no elements.
  - `max_elements` defines the maximum number of elements that can be stored in the structure(can be increased/shrunk).
  - `ef_construction` defines a construction time/accuracy trade-off (see ALGO_PARAMS.md).
  - `M` defines tha maximum number of outgoing connections in the graph (ALGO_PARAMS.md).
  - `allow_replace_deleted` enables replacing of deleted elements with new added ones.
- `add_items(data, ids, num_threads = -1, replace_deleted = False)` - inserts the `data` (numpy array of vectors, shape: `N*dim`) into the structure.
  - `num_threads` sets the number of cpu threads to use (-1 means use default).
  - `ids` are optional N-size numpy array of integer labels for all elements in `data`.
    - If index already has the elements with the same labels, their features will be updated. Note that update procedure is slower than insertion of a new element, but more memory- and query-efficient.
  - `replace_deleted` replaces deleted elements. Note it allows to save memory.
    - to use it `init_index` should be called with `allow_replace_deleted=True`
  - Thread-safe with other `add_items` calls, but not with `knn_query`.
- `mark_deleted(label)` - marks the element as deleted, so it will be omitted from search results. Throws an exception if it is already deleted.
- `unmark_deleted(label)` - unmarks the element as deleted, so it will be not be omitted from search results.
- `resize_index(new_size)` - changes the maximum capacity of the index. Not thread safe with `add_items` and `knn_query`.
- `set_ef(ef)` - sets the query time accuracy/speed trade-off, defined by the `ef` parameter (ALGO_PARAMS.md). Note that the parameter is currently not saved along with the index, so you need to set it manually after loading.
- `knn_query(data, k = 1, num_threads = -1, filter = None)` make a batch query for `k` closest elements for each element of the
  - `data` (shape: `N*dim`). Returns a numpy array of (shape: `N*k`).
  - `num_threads` sets the number of cpu threads to use (-1 means use default).
  - `filter` filters elements by its labels, returns elements with allowed ids. Note that search with a filter works slow in python in multithreaded mode. It is recommended to set `num_threads=1`
  - Thread-safe with other `knn_query` calls, but not with `add_items`.
- `load_index(path_to_index, max_elements = 0, allow_replace_deleted = False)` loads the index from persistence to the uninitialized index.
  - `max_elements` (optional) resets the maximum number of elements in the structure.
  - `allow_replace_deleted` specifies whether the index being loaded has enabled replacing of deleted elements.
- `save_index(path_to_index)` saves the index from persistence.

- `set_num_threads(num_threads)` set the default number of cpu threads used during data insertion/querying.
- `get_items(ids, return_type = 'numpy')` - returns a numpy array (shape: `N*dim`) of vectors that have integer identifiers specified in `ids` numpy vector (shape: `N`) if `return_type` is `list` return list of lists. Note that for cosine similarity it currently returns **normalized** vectors.
- `get_ids_list()` - returns a list of all elements' ids.
- `get_max_elements()` - returns the current capacity of the index
- `get_current_count()` - returns the current number of element stored in the index

Read-only properties of `hnswlib.Index` class:

- `space` - name of the space (can be one of "l2", "ip", or "cosine").
- `dim` - dimensionality of the space.
- `M` - parameter that defines the maximum number of outgoing connections in the graph.
- `ef_construction` - parameter that controls speed/accuracy trade-off during the index construction.
- `max_elements` - current capacity of the index. Equivalent to `p.get_max_elements()`.
- `element_count` - number of items in the index. Equivalent to `p.get_current_count()`.

Properties of `hnswlib.Index` that support reading and writing:

- `ef` - parameter controlling query time/accuracy trade-off.
- `num_threads` - default number of threads to use in `add_items` or `knn_query`. Note that calling `p.set_num_threads(3)` is equivalent to `p.num_threads=3`.

Python bindings examples

See more examples here:

- Creating index, inserting elements, searching, serialization/deserialization
- Filtering during the search with a boolean function
- Deleting the elements and reusing the memory of the deleted elements for newly added elements

An example of creating index, inserting elements, searching and pickle serialization:

```
import hnswlib
import numpy as np
import pickle

dim = 128
num_elements = 10000

# Generating sample data
data = np.float32(np.random.random((num_elements, dim)))
ids = np.arange(num_elements)

# Declaring index
p = hnswlib.Index(space = 'l2', dim = dim) # possible options are l2, cosine or ip

# Initializing index - the maximum number of elements should be known beforehand
p.init_index(max_elements = num_elements, ef_construction = 200, M = 16)

# Element insertion (can be called several times):
p.add_items(data, ids)

# Controlling the recall by setting ef:
p.set_ef(50) # ef should always be > k

# Query dataset, k - number of the closest elements (returns 2 numpy arrays)
labels, distances = p.knn_query(data, k = 1)

# Index objects support pickling
```

```
# WARNING: serialization via pickle.dumps(p) or p.__getstate__() is NOT thread-safe with p.add_items method!
# Note: ef parameter is included in serialization; random number generator is initialized with random_seed on Index
p_copy = pickle.loads(pickle.dumps(p)) # creates a copy of index p using pickle round-trip

### Index parameters are exposed as class properties:
print(f"Parameters passed to constructor:  space={p_copy.space}, dim={p_copy.dim}")
print(f"Index construction: M={p_copy.M}, ef_construction={p_copy.ef_construction}")
print(f"Index size is {p_copy.element_count} and index capacity is {p_copy.max_elements}")
print(f"Search speed/quality trade-off parameter: ef={p_copy.ef}")
```

An example with updates after serialization/deserialization:

```
import hnswlib
import numpy as np

dim = 16
num_elements = 10000

# Generating sample data
data = np.float32(np.random.random((num_elements, dim)))

# We split the data in two batches:
data1 = data[:num_elements // 2]
data2 = data[num_elements // 2:]

# Declaring index
p = hnswlib.Index(space='l2', dim=dim)  # possible options are l2, cosine or ip

# Initializing index
# max_elements - the maximum number of elements (capacity). Will throw an exception if exceeded
# during insertion of an element.
# The capacity can be increased by saving/loading the index, see below.
#
# ef_construction - controls index search speed/build speed tradeoff
#
# M - is tightly connected with internal dimensionality of the data. Strongly affects memory consumption (~M)
# Higher M leads to higher accuracy/run_time at fixed ef/efConstruction

p.init_index(max_elements=num_elements//2, ef_construction=100, M=16)

# Controlling the recall by setting ef:
# higher ef leads to better accuracy, but slower search
p.set_ef(10)

# Set number of threads used during batch search/construction
# By default using all available cores
p.set_num_threads(4)

print("Adding first batch of %d elements" % (len(data1)))
p.add_items(data1)

# Query the elements for themselves and measure recall:
labels, distances = p.knn_query(data1, k=1)
print("Recall for the first batch:", np.mean(labels.reshape(-1) == np.arange(len(data1))), "\n")

# Serializing and deleting the index:
index_path='first_half.bin'
print("Saving index to '%s'" % index_path)
p.save_index("first_half.bin")
del p

# Re-initializing, loading the index
p = hnswlib.Index(space='l2', dim=dim)  # the space can be changed - keeps the data, alters the distance function.

print("\nLoading index from 'first_half.bin'\n")

# Increase the total capacity (max_elements), so that it will handle the new data
p.load_index("first_half.bin", max_elements = num_elements)

print("Adding the second batch of %d elements" % (len(data2)))
p.add_items(data2)
```

```
# Query the elements for themselves and measure recall:
labels, distances = p.knn_query(data, k=1)
print("Recall for two batches:", np.mean(labels.reshape(-1) == np.arange(len(data))), "\n")
```

C++ examples

See examples here:

- creating index, inserting elements, searching, serialization/deserialization
- no-exceptions (`*NoExceptions`) variants of insert / search / I/O for `-fno-exceptions` builds
- filtering during the search with a boolean function
- deleting the elements and reusing the memory of the deleted elements for newly added elements
- multithreaded usage
- multivector search
- epsilon search

Bindings installation

You can install from sources:

```
apt-get install -y python-setuptools python-pip
git clone https://github.com/nmslib/hnswlib.git
cd hnswlib
pip install .
```

or you can install via pip: `pip install hnswlib`

For developers

Contributions are highly welcome!

Please make pull requests against the `develop` branch.

When making changes please run tests (and please add a test to `tests/python` in case there is new functionality):

```
python -m unittest discover --start-directory tests/python --pattern "bindings_test*.py"
```

Other implementations

- Non-metric space library (nmslib) - main library(python, C++), supports exotic distances: https://github.com/nmslib/nmslib
- Faiss library by facebook, uses own HNSW implementation for coarse quantization (python, C++): https://github.com/facebookresearch/faiss
- Code for the paper "Revisiting the Inverted Indices for Billion-Scale Approximate Nearest Neighbors" (current state-of-the-art in compressed indexes, C++): https://github.com/dbaranchuk/ivf-hnsw
- Amazon PECOS https://github.com/amzn/pecos
- TOROS N2 (python, C++): https://github.com/kakao/n2
- Online HNSW (C++): https://github.com/andrusha97/online-hnsw)
- Go implementation: https://github.com/Bithack/go-hnsw
- Python implementation (as a part of the clustering code by by Matteo Dell'Amico): https://github.com/matteodellamico/flexible-clustering
- Julia implmentation https://github.com/JuliaNeighbors/HNSW.jl
- Java implementation: https://github.com/jelmerk/hnswlib
- Java bindings using Java Native Access: https://github.com/stepstone-tech/hnswlib-jna
- .Net implementation: https://github.com/curiosity-ai/hnsw-sharp
- CUDA implementation: https://github.com/js1010/cuhnsw
- Rust implementation https://github.com/rust-cv/hnsw

- Rust implementation for memory and thread safety purposes and There is A Trait to enable the user to implement its own distances. It takes as data slices of types T satisfying T:Serialize+Clone+Send+Sync.: https://github.com/jean-pierreBoth/hnswlib-rs

200M SIFT test reproduction

To download and extract the bigann dataset (from root directory):

```
python tests/cpp/download_bigann.py
```

To compile:

```
mkdir build
cd build
cmake ..
make all
```

To run the test on 200M SIFT subset:

```
./main
```

The size of the BigANN subset (in millions) is controlled by the variable **subset_size_millions** hardcoded in **sift_1b.cpp**.

Updates test

To generate testing data (from root directory):

```
cd tests/cpp
python update_gen_data.py
```

To compile (from root directory):

```
mkdir build
cd build
cmake ..
make
```

To run test **without** updates (from `build` directory)

```
./test_updates
```

To run test **with** updates (from `build` directory)

```
./test_updates update
```

HNSW example demos

- Visual search engine for 1M amazon products (MXNet + HNSW): website, code, demo by @ThomasDelteil

References

HNSW paper:

```
@article{malkov2018efficient,
  title={Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs},
  author={Malkov, Yu A and Yashunin, Dmitry A},
  journal={IEEE transactions on pattern analysis and machine intelligence},
  volume={42},
  number={4},
  pages={824--836},
  year={2018},
```

```
  publisher={IEEE}
}
```

The update algorithm supported in this repository is to be published in "Dynamic Updates For HNSW, Hierarchical Navigable Small World

Releases 12

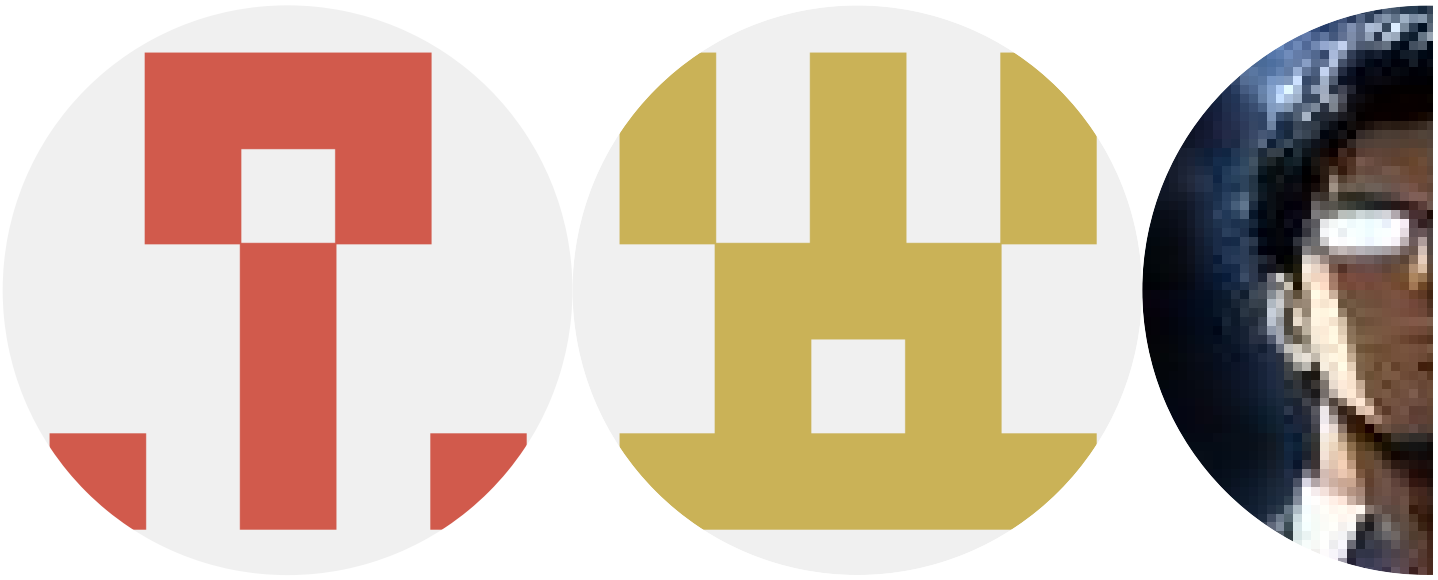
v0.9.0 Latest
7 months ago

+ 11 releases

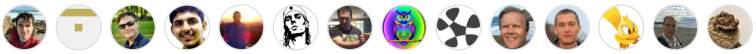
Packages

No packages published

Used by 8.7K



Contributors 54



+ 40 contributors

Languages

C++ 76.2% **Python** 19.4% **CMake** 3.7% **Other** 0.7%